Detailed Design

for

Mentor Caring System

Version 1.0 approved

Prepared by Ma Xinyao(FUll), Guan Xinling(FUll), Hou Shuoran(FUll), Peng Yancheng(FUll), Tang Jianxu(FUll), Ouyang Hao(FUll)

B09

2026.04.25

Contents

Revision History

Name	Date	Reason For Changes	Version
B09	2026.04.25	Initial approved version	1.0

1. Overview

1.1 Project description

The Mentor Caring System is a web-based supporting system for the BNBU Mentor Caring Programme. It is developed to help manage the main mentoring activities for undergraduate students in the university. The main purposes of the project are to allocate mentors, report interview records, make appointments, generate summary reports, and track related information.

1.2 References

Software Requirements Specification for Mentor Caring System, Version 1.4 approved.

UI Diagram.pdf, User Interface document for Mentor Caring System.

Design and Implementation of a Mentor Caring System, Project Long Description, Version 1.2, 2025–2026.

1.3 Design purpose

The detailed design adopts the Divide and Conquer approach. This approach is suitable because the Mentor Caring System includes multiple user roles and several independent functions.

By decomposing the system into smaller parts according to responsibilities, the design reduces complexity and improves modularity, maintainability, and scalability. It also makes future extension and modification easier.

2. Overall description

2.1 Class diagram

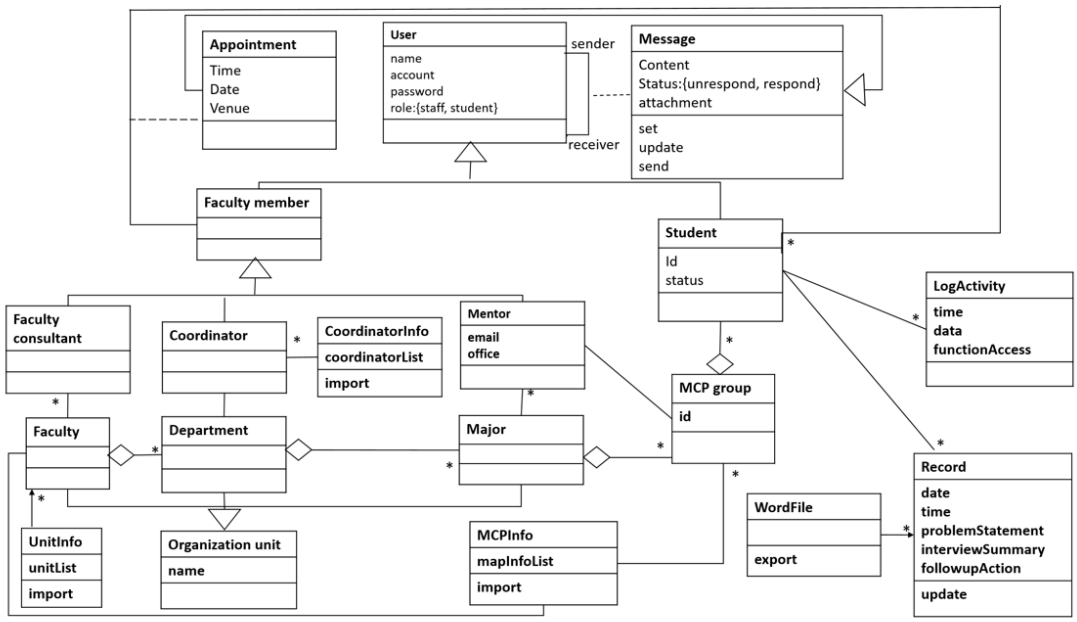

Figure 1: Class Diagram of the Mentor Caring System

2.2 Refinements

In the detailed design, the original class diagram is refined to better support implementation. The main refinement work includes identifying system features, restructuring classes into subsystems, clarifying responsibilities, and adding UI, control, persistent, and database-related classes.

The system is divided into three main subsystems: Common User Subsystem, Organization and Group Allocation Subsystem, and Mentoring Operation Subsystem. This reduces complexity and improves modularity, maintainability, and scalability.

Important constraints are also made clearer, such as role-based access control, preservation of historical records, and integration with external resources.

3. Detailed design

This section describes the classes after the detailed design. Compared with the class diagram in Section 2.1, the refined class diagram gives a more detailed and implementation-oriented structure of the Mentor Caring System.

3.1 Class diagram

The refined class diagram is shown in Figure 2. Compared with the class diagram in the architecture design, this refined version provides more implementation-level details, including attributes, operation signatures, inheritance relationships, association relationships, multiplicities, and import-related helper classes.

First, the user hierarchy is refined more clearly. `User` is used as the common parent class for system users. `Supporting staff`, `Administrator`, `Faculty member`, and `Student` are modeled as specialized classes of `User`. In addition, `Faculty consultant`, `Coordinator`, and `Mentor` are modeled as specialized classes of `Faculty member`, because they are faculty-related roles in the Mentor Caring Programme.

Second, the communication and appointment arrangement design is refined. `Message` is modeled as the core communication class. It records the message content, optional attachment, and message status. In the refined class diagram, `Message` is associated with `User` twice. One association represents the sender, and the other association represents the receiver. The multiplicities show that each message has one sender and one receiver. The labels `messageId` and `userId` are used as implementation identifiers for maintaining the associations between `Message` and `User`. `Appointment` is modeled as a specialized class of `Message`. It records appointment-specific information, including appointment date, time, venue, appointment status, `iniatorId`, and `attendeeId`. In the refined class diagram, each `Appointment` is associated with one `Faculty member` and one `Student`. The multiplicities shown in the diagram are `1` on both sides of these associations.

Third, the organization-related structure is refined through `Organization unit`, `Faculty`, `Department`, `Major`, and `MCP group`. `Organization unit` provides a common abstraction for organization-related classes. In the refined class diagram, `Faculty`, `Department`, and `Major` are modeled as specialized classes of `Organization unit`. Therefore, they inherit the common organization attributes `unitId` and `name`, as well as the operations `getName()` and `getId()`. `Faculty`, `Department`, and `Major` represent the academic organization structure, while `MCP group` represents the mentor-group allocation structure.

Finally, the refined class diagram also makes the relationships among `Student`, `LogActivity`, `Record`, and `WordFile` more explicit. A `Student` generates `LogActivity`. The association is labeled with `LogID`, and the multiplicity is shown as `1` on both sides. The relationship between `Student` and `Record` is labeled with `RecordID`, with multiplicity `1` on both sides. The relationship between `Record` and `WordFile` is also labeled with `RecordID`, with multiplicity `1` on both sides. Other labels such as `appointmentId`, `FacultyConsultantId`, `FacultyId`, `DepartmentId`, `CoordinatorID`, `MajorID`, `MentorID`, `MCPGroupID`, `StudentID`, and `GroupID` indicate implementation identifiers used to maintain associations between related classes. The helper classes `UnitInfo`, `CoordinatorInfo`, and `MCPInfo` store imported information as lists.

3.2 Classes

This subsection gives the details of the main classes in the refined class diagram.

3.2.1 User

```
User
--------------------------------
```

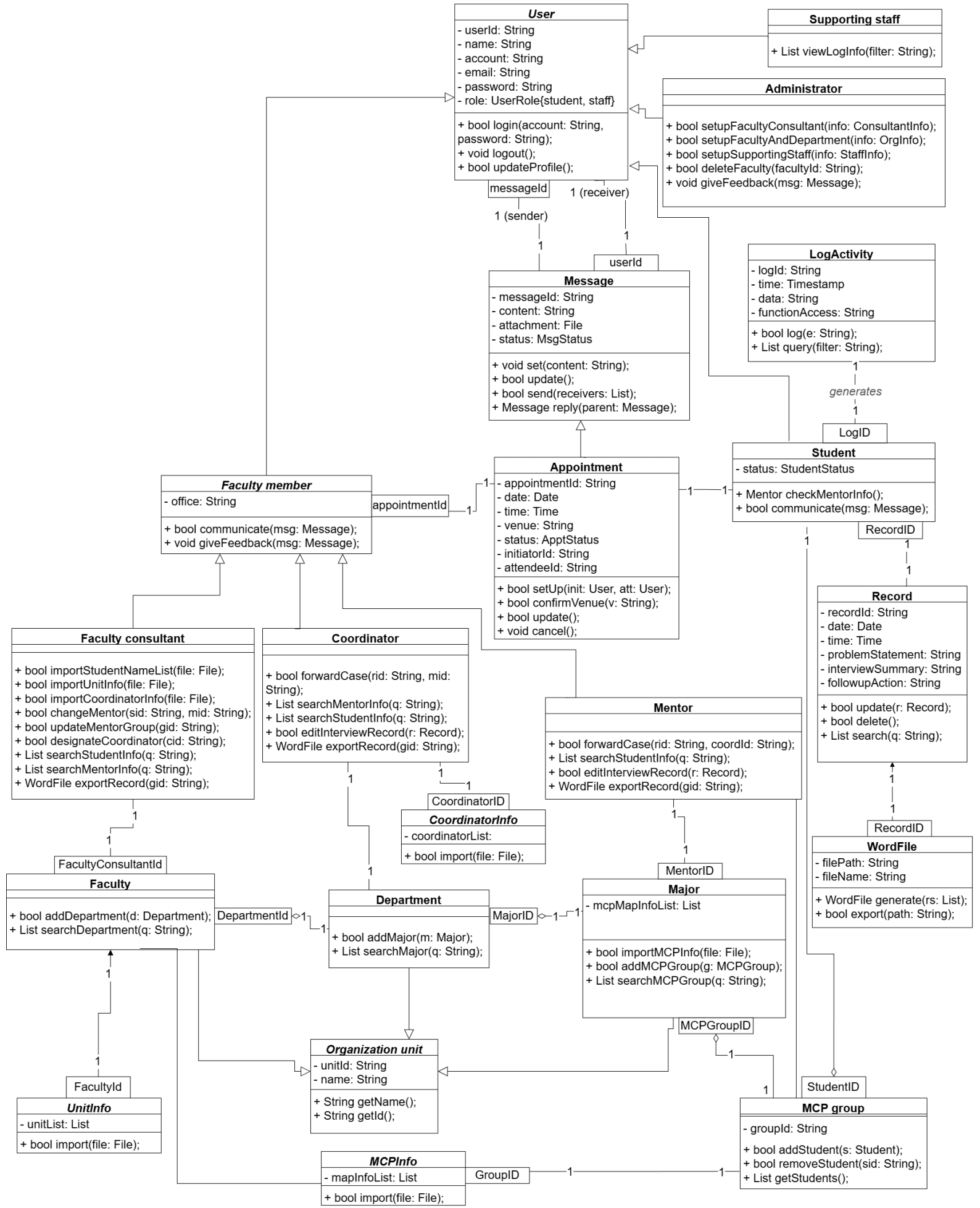

Figure 2: Class Diagram of the Mentor Caring System

```
- userId: String
- name: String
- account: String
- email: String
- password: String
- role: UserRole{student, staff}
---------------------------------
+ bool login(account: String, password: String);
+ void logout();
+ bool updateProfile();
```

Explanations `User` is the common parent class of all system users. The attribute `userId` uniquely identifies a user. The attributes `name`, `account`, `email`, `password`, and `role` store the user's basic authentication, contact, and role information. The role is represented by `UserRole{student, staff}` in the refined class diagram. The operations `login()`, `logout()`, and `updateProfile()` provide common user functions for inherited user roles.

3.2.2 Supporting staff

```
Supporting staff
---------------------------------
---------------------------------
+ List viewLogInfo(filter: String);
```

Explanations `Supporting staff` represents the system maintenance role. In the refined class diagram, `Supporting staff` is modeled as a specialized class of `User`. It does not define additional attributes because basic user information is inherited from `User`. The operation `viewLogInfo()` allows supporting staff to view log information according to a filter condition.

3.2.3 Administrator

```
Administrator
---------------------------------
---------------------------------
+ bool setupFacultyConsultant(info: ConsultantInfo);
+ bool setupFacultyAndDepartment(info: OrgInfo);
+ bool setupSupportingStaff(info: StaffInfo);
+ bool deleteFaculty(facultyId: String);
+ void giveFeedback(msg: Message);
```

Explanations `Administrator` is responsible for system-level setup and maintenance. In the refined class diagram, `Administrator` is modeled as a specialized class of `User`. It does not define additional attributes because basic user information is inherited from

`User`. The operation `setupFacultyConsultant()` is used to set up faculty consultant information. The operation `setupFacultyAndDepartment()` is used to set up faculty and department information. The operation `setupSupportingStaff()` is used to set up supporting staff information. The operation `deleteFaculty()` deletes a faculty according to the given faculty identifier. The operation `giveFeedback()` allows the administrator to give feedback through a message.

3.2.4 Faculty member

```
Faculty member
---------------------------------
- office: String
---------------------------------
+ bool communicate(msg: Message);
+ void giveFeedback(msg: Message);
```

Explanations `Faculty member` is the parent class of faculty-related roles. In the refined class diagram, `Faculty member` is modeled as a specialized class of `User`. The basic user information is inherited from `User`. The attribute `office` stores the office information of the faculty member. The operation `communicate()` allows a faculty member to communicate through a message. The operation `giveFeedback()` allows a faculty member to give feedback through a message.

3.2.5 Faculty consultant

```
Faculty consultant
---------------------------------
---------------------------------
+ bool importStudentNameList(file: File);
+ bool importUnitInfo(file: File);
+ bool importCoordinatorInfo(file: File);
+ bool changeMentor(sid: String, mid: String);
+ bool updateMentorGroup(gid: String);
+ bool designateCoordinator(cid: String);
+ List searchStudentInfo(q: String);
+ List searchMentorInfo(q: String);
+ WordFile exportRecord(gid: String);
```

Explanations `Faculty consultant` manages student, mentor, and coordinator-related information within a faculty. In the refined class diagram, `Faculty consultant` is modeled as a specialized class of `Faculty member`. It does not define additional attributes in its class notation. The association between `Faculty consultant` and `Faculty` is represented by the identifier label `FacultyConsultantId`, with multiplicity `1` on both sides.

The operation `importStudentNameList()` imports student name list information from a file. The operation `importUnitInfo()` imports unit information from a file. The

operation `importCoordinatorInfo()` imports coordinator information from a file. The operation `changeMentor()` changes the mentor according to the student identifier and mentor identifier. The operation `updateMentorGroup()` updates a mentor group according to the group identifier. The operation `designateCoordinator()` designates a coordinator according to the coordinator identifier. The operations `searchStudentInfo()` and `searchMentorInfo()` search student and mentor information. The operation `exportRecord()` exports records to a Word file according to the group identifier.

Pre- and post-conditions for `importStudentNameList(file: File)`:

- **Pre-condition:** The input file must exist, must be readable, and must follow the required format for importing student name list information.
- **Post-condition:** Valid student name list information is imported into the system, and related student, mentor, and group allocation information can be maintained by the system.

3.2.6 Coordinator

```
Coordinator
--------------------------------
--------------------------------
+ bool forwardCase(rid: String, mid: String);
+ List searchMentorInfo(q: String);
+ List searchStudentInfo(q: String);
+ bool editInterviewRecord(r: Record);
+ WordFile exportRecord(gid: String);
```

Explanations `Coordinator` represents a coordinator in the Mentor Caring System. In the refined class diagram, `Coordinator` is modeled as a specialized class of `Faculty member`. It does not define additional attributes in its class notation. The association between `Coordinator` and `Department` is represented by the identifier label `CoordinatorID`, with multiplicity `1` on both sides. The association between `Coordinator` and `CoordinatorInfo` is also represented by the identifier label `CoordinatorID`, with multiplicity `1` on both sides.

The operation `forwardCase()` forwards a case according to the record identifier and mentor identifier. The operation `searchMentorInfo()` searches mentor information. The operation `searchStudentInfo()` searches student information. The operation `editInterviewRecord()` edits an interview record. The operation `exportRecord()` exports records to a Word file according to the group identifier.

3.2.7 Mentor

```
Mentor
--------------------------------
--------------------------------
+ bool forwardCase(rid: String, coordId: String);
+ List searchStudentInfo(q: String);
```

```
+ bool editInterviewRecord(r: Record);
+ WordFile exportRecord(gid: String);
```

Explanations `Mentor` represents a mentor in the Mentor Caring System. In the refined class diagram, `Mentor` is modeled as a specialized class of `Faculty member`. It does not define additional attributes in its class notation. The association between `Mentor` and `Major` is represented by the identifier label `MentorID`, with multiplicity `1` on both sides.

The operation `forwardCase()` forwards a case according to the record identifier and coordinator identifier. The operation `searchStudentInfo()` searches student information. The operation `editInterviewRecord()` edits an interview record. The operation `exportRecord()` exports records to a Word file according to the group identifier.

3.2.8 Student

```
Student
--------------------------------
- status: StudentStatus
--------------------------------
+ Mentor checkMentorInfo();
+ bool communicate(msg: Message);
```

Explanations `Student` represents a student in the Mentor Caring System. In the refined class diagram, `Student` is modeled as a specialized class of `User`. The basic user information is inherited from `User`. The attribute `status` records the student's status. The relationship between `Student` and `LogActivity` is represented through the association labeled `LogID`, with multiplicity `1` on both sides. The relationship between `Student` and `Record` is represented through the association labeled `RecordID`, with multiplicity `1` on both sides. The relationship between `Student` and `MCP group` is represented through the association labeled `StudentID`, with multiplicity `1` on both sides. The relationship between `Student` and `Appointment` shows that each appointment is associated with one student.

The operation `checkMentorInfo()` allows the student to check mentor information. The operation `communicate()` allows the student to communicate through a message.

3.2.9 Message

```
Message
--------------------------------
- messageId: String
- content: String
- attachment: File
- status: MsgStatus
--------------------------------
+ void set(content: String);
+ bool update();
```

```
+ bool send(receivers: List);
+ Message reply(parent: Message);
```

Explanations `Message` is the core communication class. The attribute `messageId` uniquely identifies a message. The attribute `content` stores the message content. The attribute `attachment` stores an attached file. The attribute `status` stores the message status.

In the refined class diagram, `Message` is associated with `User` twice. One association represents the sender, and the other association represents the receiver. Each message has one sender and one receiver. The association labels `messageId` and `userId` are used as implementation identifiers for maintaining the relationship between `Message` and `User`.

The operation `set()` sets the message content. The operation `update()` updates the message. The operation `send()` sends the message to receivers. The operation `reply()` creates a reply message based on a parent message.

3.2.10 Appointment

```
Appointment
--------------------------------
- appointmentId: String
- date: Date
- time: Time
- venue: String
- status: ApptStatus
- iniatorId: String
- attendeeId: String
--------------------------------
+ bool setUp(init: User, att: User);
+ bool confirmVenue(v: String);
+ bool update();
+ void cancel();
```

Explanations `Appointment` represents an appointment arrangement. In the refined class diagram, `Appointment` is modeled as a specialized class of `Message`. The attribute `appointmentId` uniquely identifies an appointment. The attribute `date` stores the appointment date. The attribute `time` stores the appointment time. The attribute `venue` stores the appointment venue. The attribute `status` stores the appointment status. The attributes `iniatorId` and `attendeeId` record the users involved in the appointment workflow.

In the refined class diagram, `Appointment` is associated with `Faculty member` through the association labeled `appointmentId`, with multiplicity `1` on both sides. It is also associated with `Student`, with multiplicity `1` on both sides. Each appointment is associated with one faculty member and one student.

The operation `setUp()` sets up an appointment between the initiator and attendee. The operation `confirmVenue()` confirms the appointment venue. The operation `update()` updates the appointment. The operation `cancel()` cancels the appointment.

3.2.11 LogActivity

```
LogActivity
--------------------------------
- logId: String
- time: Timestamp
- data: String
- functionAccess: String
--------------------------------
+ bool log(e: String);
+ List query(filter: String);
```

Explanations `LogActivity` records system activity information. The attribute `logId` uniquely identifies a log activity. The attribute `time` stores the timestamp of the activity. The attribute `data` stores related data. The attribute `functionAccess` stores the accessed function.

In the refined class diagram, a `Student` generates `LogActivity`. The relationship between `Student` and `LogActivity` is represented through the association labeled `LogID`, with multiplicity `1` on both sides. The operation `log()` records an event. The operation `query()` queries log activities according to a filter condition.

3.2.12 Record

```
Record
--------------------------------
- recordId: String
- date: Date
- time: Time
- problemStatement: String
- interviewSummary: String
- followupAction: String
--------------------------------
+ bool create();
+ bool update(r: Record);
+ bool delete();
+ List search(q: String);
```

Explanations `Record` represents an interview record. The attribute `recordId` uniquely identifies a record. The attribute `date` stores the record date. The attribute `time` stores the record time. The attribute `problemStatement` stores the problem statement. The at-

tribute `interviewSummary` stores the interview summary. The attribute `followupAction` stores the follow-up action.

In the refined class diagram, the relationship between `Student` and `Record` is represented through the association labeled `RecordID`, with multiplicity `1` on both sides. The relationship between `Record` and `WordFile` is also represented through the association labeled `RecordID`, with multiplicity `1` on both sides.

The operation `create()` creates a record. The operation `update()` updates a record. The operation `delete()` deletes a record. The operation `search()` searches records according to a query condition.

3.2.13 WordFile

```
WordFile
---------------------------------
- filePath: String
- fileName: String
---------------------------------
+ WordFile generate(rs: List);
+ bool export(path: String);
```

Explanations `WordFile` represents an exported Word file. The attribute `filePath` stores the file path. The attribute `fileName` stores the file name. In the refined class diagram, `WordFile` is associated with `Record` through the association labeled `RecordID`, with multiplicity `1` on both sides. The operation `generate()` generates a Word file from a list of records. The operation `export()` exports the Word file to a path.

3.2.14 Organization unit

```
Organization unit
---------------------------------
- unitId: String
- name: String
---------------------------------
+ String getName();
+ String getId();
```

Explanations `Organization unit` is the parent class of organization-related classes. The attribute `unitId` uniquely identifies an organization unit. The attribute `name` stores the organization unit name. The operation `getName()` returns the organization unit name. The operation `getId()` returns the organization unit identifier. In the refined class diagram, `Faculty`, `Department`, and `Major` are modeled as specialized classes of `Organization unit`.

3.2.15 Faculty

```
Faculty
--------------------------------
--------------------------------
+ bool addDepartment(d: Department);
+ List searchDepartment(q: String);
```

Explanations `Faculty` represents a faculty. In the refined class diagram, `Faculty` is modeled as a specialized class of `Organization unit`. Therefore, it inherits `unitId`, `name`, `getName()`, and `getId()` from `Organization unit`. The relationship between `UnitInfo` and `Faculty` is represented through the association labeled `FacultyId`, with multiplicity `1` on both sides. The relationship between `Faculty consultant` and `Faculty` is represented through the association labeled `FacultyConsultantId`, with multiplicity `1` on both sides. The relationship between `Faculty` and `Department` is represented through the association labeled `DepartmentId`, with multiplicity `1` on both sides. The operation `addDepartment()` adds a department. The operation `searchDepartment()` searches departments according to a query condition.

3.2.16 Department

```
Department
--------------------------------
--------------------------------
+ bool addMajor(m: Major);
+ List searchMajor(q: String);
```

Explanations `Department` represents a department. In the refined class diagram, `Department` is modeled as a specialized class of `Organization unit`. Therefore, it inherits `unitId`, `name`, `getName()`, and `getId()` from `Organization unit`. The relationship between `Faculty` and `Department` is represented through the association labeled `DepartmentId`, with multiplicity `1` on both sides. The relationship between `Department` and `Coordinator` is represented through the association labeled `CoordinatorID`, with multiplicity `1` on both sides. The relationship between `Department` and `Major` is represented through the association labeled `MajorID`, with multiplicity `1` on both sides. The operation `addMajor()` adds a major. The operation `searchMajor()` searches majors according to a query condition.

3.2.17 Major

```
Major
--------------------------------
- mcpMapInfoList: List
--------------------------------
+ bool importMCPInfo(file: File);
```

```
+ bool addMCPGroup(g: MCPGroup);
+ List searchMCPGroup(q: String);
```

Explanations `Major` represents a major. In the refined class diagram, `Major` is modeled as a specialized class of `Organization unit`. Therefore, it inherits `unitId`, `name`, `getName()`, and `getId()` from `Organization unit`. The attribute `mcpMapInfoList` stores MCP mapping information as a list. The relationship between `Department` and `Major` is represented through the association labeled `MajorID`, with multiplicity `1` on both sides. The relationship between `Major` and `Mentor` is represented through the association labeled `MentorID`, with multiplicity `1` on both sides. The relationship between `Major` and `MCP group` is represented through the association labeled `MCPGroupID`, with multiplicity `1` on both sides.

The operation `importMCPInfo()` imports MCP information from a file. The operation `addMCPGroup()` adds an MCP group. The operation `searchMCPGroup()` searches MCP groups according to a query condition.

3.2.18 MCP group

```
MCP group
--------------------------------
- groupId: String
--------------------------------
+ bool addStudent(s: Student);
+ bool removeStudent(sid: String);
+ List getStudents();
```

Explanations `MCP group` represents an MCP group. The attribute `groupId` uniquely identifies the MCP group. The relationship between `Major` and `MCP group` is represented through the association labeled `MCPGroupID`, with multiplicity `1` on both sides. The relationship between `MCP group` and `Student` is represented through the association labeled `StudentID`, with multiplicity `1` on both sides. The relationship between `MCPInfo` and `MCP group` is represented through the association labeled `GroupID`, with multiplicity `1` on both sides. The operation `addStudent()` adds a student to the group. The operation `removeStudent()` removes a student according to the student identifier. The operation `getStudents()` returns the students in the group.

3.2.19 UnitInfo

```
UnitInfo
--------------------------------
- unitList: List
--------------------------------
+ bool import(file: File);
```

Explanations `UnitInfo` is an import helper class for unit information. The attribute `unitList` stores imported unit information as a list. The operation `import()` imports unit information from a file. The relationship between `UnitInfo` and `Faculty` is represented through the association labeled `FacultyId`, with multiplicity `1` on both sides.

3.2.20 CoordinatorInfo

```
CoordinatorInfo
--------------------------------
- coordinatorList: List
--------------------------------
+ bool import(file: File);
```

Explanations `CoordinatorInfo` is an import helper class for coordinator information. The attribute `coordinatorList` stores imported coordinator information as a list. The operation `import()` imports coordinator information from a file. The relationship between `CoordinatorInfo` and `Coordinator` is represented through the association labeled `CoordinatorID`, with multiplicity `1` on both sides.

3.2.21 MCPInfo

```
MCPInfo
--------------------------------
- mapInfoList: List
--------------------------------
+ bool import(file: File);
```

Explanations `MCPInfo` is an import helper class for MCP mapping information. The attribute `mapInfoList` stores imported MCP mapping information as a list. The operation `import()` imports MCP mapping information from a file. The relationship between `MCPInfo` and `MCP group` is represented through the association labeled `GroupID`, with multiplicity `1` on both sides.

4. More considerations

Several considerations should be noted in the implementation of this design.

First, inheritance should be implemented consistently with the refined class diagram. `Supporting staff`, `Administrator`, `Faculty member`, and `Student` inherit from `User`. `Faculty consultant`, `Coordinator`, and `Mentor` inherit from `Faculty member`. `Appointment` inherits from `Message`. `Faculty`, `Department`, and `Major` inherit from `Organization unit`.

Second, association identifiers should be handled carefully in implementation. Labels such as `messageId`, `userId`, `appointmentId`, `LogID`, `RecordID`, `FacultyConsultantId`, `FacultyId`, `DepartmentId`, `CoordinatorID`, `MajorID`, `MentorID`, `MCPGroupID`, `StudentID`,

and `GroupID` are used to maintain relationships between related classes. These identifiers should be kept consistent when related objects are created, updated, searched, imported, exported, or deleted.

Third, the import helper classes should follow the refined class diagram. `UnitInfo` stores `unitList: List`, `CoordinatorInfo` stores `coordinatorList: List`, and `MCPInfo` stores `mapInfoList: List`. Their data should be imported through `import(file: File)`.

Fourth, appointment handling should follow the inheritance and association design. Since `Appointment` is a specialized class of `Message`, it should be handled as a communication-related object. At the same time, it stores appointment-specific information such as `date`, `time`, `venue`, `status`, `iniatorId`, and `attendeeId`. Each appointment should be linked with one related student and one related faculty member according to the associations in the class diagram.

Fifth, record export should follow the relationship between `Record` and `WordFile`. `Record` stores interview-related information, and `WordFile` stores file export information. The association labeled `RecordID` should be used to maintain the one-to-one relationship between records and exported Word files.

Finally, system maintenance operations should preserve relationship consistency. For example, when `deleteFaculty(facultyId: String)` is executed by `Administrator`, related associations involving `Faculty`, `Department`, `Faculty consultant`, and `UnitInfo` should be checked to avoid inconsistent organization information.